

UUID [SimTalk]

Returns the *UUID* (Universally Unique Identifier) of the object designated by <Path> which it keeps during its entire lifetime.

Type

Read-only attribute

Syntax

```
<Path>.UUID → string
```

Return Value

The return value has the data type *string*.

Example

```
print Station.UUID  
// returns 3daf7c80-9411-4671-966f-f3ecd2c0f478 for example
```

See also

[What is a Folder Model?](#)

Attributes of All Objects

Attributes of All Objects

The objects in the *Class Library* have predefined attributes controlling their behavior or representing their state.

- In the dialog windows of the objects you select or enter settings with the check boxes, the text boxes and the drop-down lists on the tabs of the object.
- In *SimTalk* you can assign values to the respective attributes.

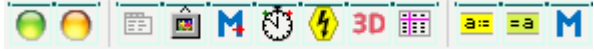
For most **attributes** you can set and get their values.

- To **set the value** of an attribute, you might, for example, type:

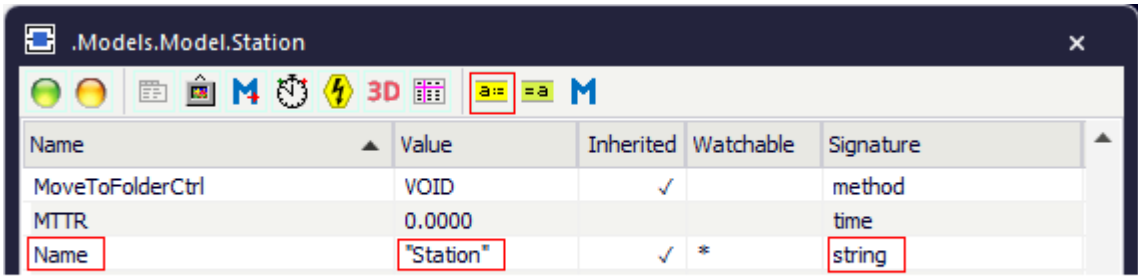
```
MyStation.Pause := true
```

- To **get the value** of an attribute, you might, for example, type:

```
print MyStation.Pause
posit := MyStation.Cont.XPos
```

Some attributes are **read-only**, i.e., you can only get (query) their value. You can filter the displayed *attributes* and *methods* with the buttons on the toolbar . The filter  shows the read-only attributes.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



Name	Value	Inherited	Watchable	Signature
MoveToFolderCtrl	VOID	✓		method
MTTR	0.0000			time
Name	"Station"	✓	*	string

Name of the attribute Current value of the attribute Data type of the assignment value

You can:

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class** [\[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance** [\[general description\]](#).

See also

[_General Attributes](#)

[_Attributes of the Controls](#)

[_Attributes of User-defined Attributes](#)

Data Held in Tabular Form in Attributes [material flow objects]

Plant Simulation does not save the data of most of the attributes of data type *table* in a full-fledged table internally.

If you get the value of such an attribute, the internally held data is copied to a table and *Plant Simulation* returns this table. If you change the returned table, the internal data is not changed! To change the internal data of the attribute, assign such a table to the attribute. Then the data is copied to the attribute.

This applies, for example, to the lists and tables on the tab **Exit** of the material flow objects, to the tabs **Exit Strategy** and **Entry Strategy** of the *FlowControl* and to a number of other tables, which you can open by clicking the button **Open List/Open Table**.

Example

```
var t:table[string,string,integer]
t := MyStation.ExitStrategyMUAttributeList
t.appendRow("Name", "A", 1)           // the internal data remains
unchanged                             unchanged
MyStation.ExitStrategyMUAttributeList := t // writes data back to table
```

General Attributes

_General Attributes

The objects provide the general *attributes* listed in the table of contents to the left.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.

Name	Value	Inherited	Watchable	Signature
MoveToFolderCtrl	VOID	✓		method
MTTR	0.0000			time
Name	Station	✓	*	string

Name of the attribute Current value of the attribute Data type of the assignment value

You can:

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class [general description]**.

- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

You can set the value of an attribute and you can get its value, either with the check boxes, the text boxes and drop-down lists in the dialog windows or by assigning values to the respective attributes.

- To **set the value** of an attribute, you might, for example, type:

```
MyStation.Pause := true
```

- To **get the value** of an attribute, you might, for example, type:

```
print MyStation.Pause  
posit := MyStation.Cont.XPos
```

Coordinate3D [SimTalk] - general description

Sets the 3D coordinate of the object designated by <Path>.

Remarks

The attribute is not inherited.

You cannot query the value of *Coordinate3D* for *MU* instances.

Type

Attribute

Syntax

```
<Path>.Coordinate3D:array
```

Assignment Value

You can assign an *array* of data type *length* with three values.

Example

```
MyStation.Coordinate3D := [-10, 11, 0]  
print MyStation.Coordinate3D // returns [-10, 11, 0]
```

SimTalk

[_3D.inheritAttribute \[SimTalk\]](#)

[_3D.getPositionOfObject \[SimTalk\]](#)

[_3D.getWorldCoordinate \[SimTalk\]](#)

[Coordinate3D \[SimTalk\] - MUs](#)

CreateIn3D [SimTalk]

Sets if *Plant Simulation* creates the object designated by <Path> in 3D (**true**) or not (**false**) .

Remarks

By default, we activated this feature for all simulation-relevant objects, such as the *material flow objects*, the *fluid objects*, the *moving objects*, the *resource objects*, and the *tools*.

When you then insert an instance of this object into the **3D Scene**, it shows the icon of the object.

Note

If you deactivate the attribute for an object class and reactivate it again, *Plant Simulation* restores the built-in state of the object. It discards all changes which you made in the class of the 3D simulation object.

Note

If the attribute *CreateIn3D* has the value **false** in the origin of an object, then the attribute *CreateIn3D* also has the value **false** in the object itself and cannot be set to **true**.

Type

Attribute

Syntax

```
<Path>.CreateIn3D:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStation.CreateIn3D := true
```

See also

[Show Non-Simulation-Relevant Objects](#)

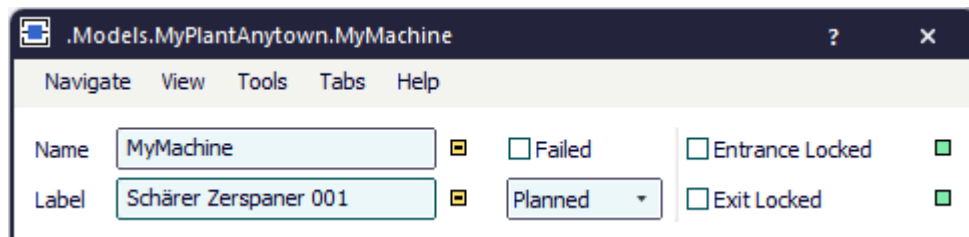
Label [SimTalk]

Sets the label of the object designated by <Path>.

Remarks

You can type in letters, numbers, blanks, and special characters.

Use the label to display objects with the same functionality with the same identifier in the *Frame*. The **Name** of the object, on the other hand, has to be unique!



Plant Simulation shows the label when you select **Options > Show Object Labels** on the **View** ribbon tab of the *Frame*.

Plant Simulation shows the label which you enter for the objects *Report*, *Dialog*, *AttributeExplorer*, and *Chart* in the title bar of their display windows. If you do not enter a label, the title bar shows the name of the object.

The same applies to a *toolbar* you add to the *Toolbox*. Select **New > Toolbar**, then select the *toolbar* in the structure, press **F2** and enter a **Label** into the dialog **Rename**.

Type

Attribute

Syntax

```
<Path>.Label:string
```

Watchable

The attribute is **watchable**.

Assignment Value

You can assign a value of data type *string*.

Example

```
print MyStation.Label  
Station1.Label := "Lathe left"
```

See also

[Label \[general description\]](#)

Name [SimTalk] - general description

Sets the name of the object designated by <Path>.

Remarks

The name has to be unique within the **name space** of the *Frame*.

You can specify a combination of letters, digits, and underscore (_) characters, for example `MyStation`, `MyStation1`, or `My_Station_1`.

The name of an object cannot start with a digit, i.e., you cannot specify `1Station`.

Note

You cannot change the name of the objects *EventController* and *Connector*.

Note

After renaming the object, you can only address it with the new name.

Type

Attribute

Syntax

```
<Path>.Name:string
```

Watchable

The attribute is **watchable**.

Assignment Value

You can assign a value of data type *string*.

Example

```
print MyStation.Name  
Station1.name := "shaft"
```

SimTalk

[setName \[SimTalk\]](#)

[isNameUnique \[SimTalk\] - identifier](#)

See also

[Name \[general description\]](#)

[Namespace](#)

RootFolder [SimTalk] - attribute

Sets if the anonymous identifier *rootFolder* takes the folder designated by <Path> into account when searching the *Class Library* (*true*) or not (*false*).

Remarks

RootFolder sets which folder *Plant Simulation* uses as the root folder. *Plant Simulation* adds an upper-case R to the icon of the folder  in the *Class Library*.

You can also set the *RootFolder* in the window **Show Attributes and Methods**.

Type

Attribute

Syntax

```
<Path>.RootFolder : boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
.ApplicationObjects.Transport.RootFolder := true
```

See also

[Set the Root Folder for Your Simulation Model](#)

[rootfolder \[SimTalk\] - anonymous identifier](#)

Attributes of the Controls

_Attributes of the Controls

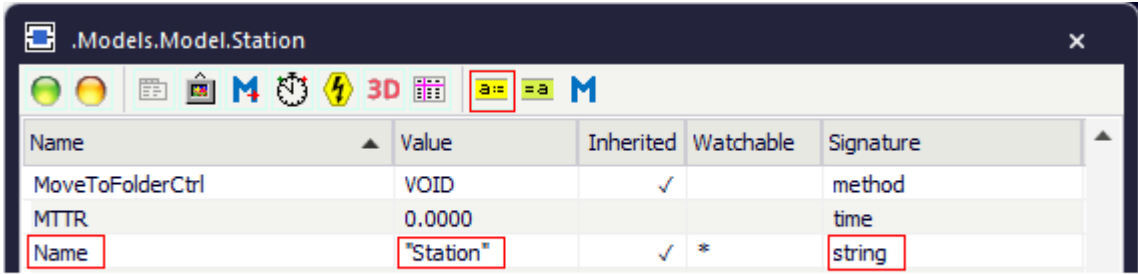
Most objects provide some or all of the *attributes* listed in the table of contents to the left to access **control methods**.

The data type of the method, which you assign, is *method*.

The expressions `&Method` or `PathToMethod.&Method` both assign an object reference to a method. Use a *string* to assign the relative path to a method. Specify `void` to deactivate a control.

Not all objects provide all controls.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



Name	Value	Inherited	Watchable	Signature
MoveToFolderCtrl	VOID	✓		method
MTTR	0.0000			time
Name	"Station"	✓	*	string

Name of the attribute Current value of the attribute Data type of the assignment value

You can:

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class** [general description].
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance** [general description].

You can set the value of an attribute and you can get its value, either with the check boxes, the text boxes and drop-down lists in the dialog windows or by assigning values to the respective attributes.

To **set the value** of an attribute, you might, for example, type:

```
MyStation.Pause := true
```

To **get the value** of an attribute, you might, for example, type:

```
print MyStation.Pause
posit := MyStation.Cont.XPos
```

See also

[Edit Controls \[material flow objects\]](#)

AvailableCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* whenever the state of the *Exporter* or the *Worker* designated by <Path> changes to **available**.

Within the control you can use the anonymous identifier ? to access the object that triggered the method.

Type

Attribute

Syntax

```
<Path>.AvailableCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyExporter.AvailableCtrl := &myAvailableCtrl
```

See also

[Available Control \[Exporter\]](#)

ChangePathCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation executes the **Change Path Control**:

- When the name of the *Folder* or the *Frame* designated by <Path> changes. The control is called for all objects contained in the *Folder* or *Frame*. The anonymous identifier @ points to the renamed object.
- When you move the *Folder* or the *Frame* designated by <Path>. The control is called for all objects contained in the *Folder* or *Frame*.

Within the *Method* you can access the moved object with the anonymous identifier question mark ?.

Note

Not each *Move* action actually moves the object. When you move an object from one *Frame* to another, *Plant Simulation* deletes the object from its source *Frame* and creates a new object in the target *Frame*. Specify an empty string "" as the location to interpret this as the top level in the hierarchy of objects, as the object **basis**.

Type

Attribute

Syntax

```
<Path>.ChangePathCtrl:method
```

Parameters

Plant Simulation passes three parameters to the *Method* that is called:

- The parameter *oldPath* of data type *string* designates the previous location of the object.
- The parameter *newPath* of data type *string* designates the new location of the object.
- The parameter *manually* of data type *boolean* indicates if you change the path of the object manually (true) or with a *Method* (false).

A **change path control** might look like this:

```
param oldPath,newPath: string, manually: boolean
print oldPath
```

```
print newPath  
print manually
```

Assignment Value

You can assign a value of data type *method*.

Example

```
ParallelStation.ChangePathCtrl := &myChangePathCtrl
```

See also

[Change Path Control](#)

CloseCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* when you close the *window* of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>.CloseCtrl:method
```

Parameter

Plant Simulation passes a parameter of data type *boolean* to the *Method* that is called. The parameter shows if the contents of the object window has changed or not.

Assignment Value

You can assign a value of data type *method*.

Example

```
MyFrame.CloseCtrl := &myCloseCtrl
```

See also

Close Control

ConnectCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* when you connect the object designated by <Path> with another object with a **Connector** and when you delete the connection between two objects.

Note

When you insert a *Connector*, *Plant Simulation* first calls the **Connect Control** for the *Connector* before calling the **Connect Controls** for the objects you are connecting.

When you delete a *Connector*, *Plant Simulation* first calls the *Connect* control for the connected objects before calling the **Destructor Control**.

Type

Attribute

Syntax

```
<Path>.ConnectCtrl:method
```

Parameters

The method has to define three parameters:

- The parameter *predecessor* of data type *object* designates the source object where the connection starts.
- The parameter *successor* of data type *object* designates the destination object where the connection ends.
- The parameter *connector* of data type *object* indicates if the connection is going to be created or deleted. If the *Connector* is going to be created, it is assigned to the parameter. If the *Connector* is going to be deleted, VOID is assigned.

Example

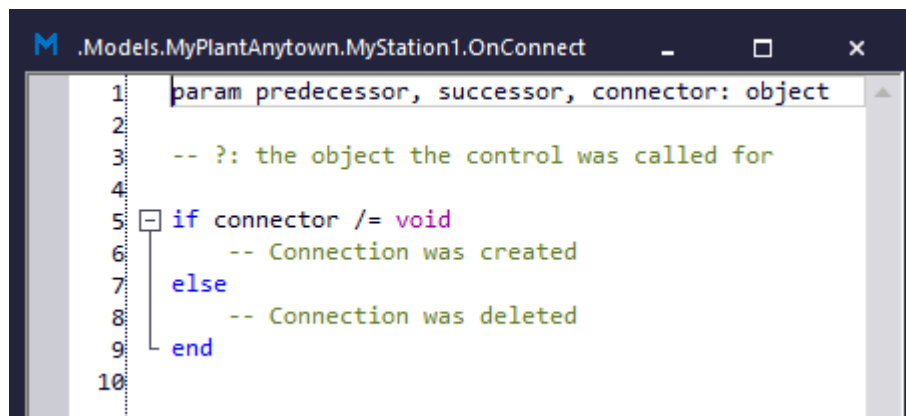
```
param ConnectionStart, ConnectionEnd: object,
ConnectionEstablishedDeleted: boolean
if ConnectionEstablishedDeleted
    print "New"
else
    print "Deleted"
end
```

```
    print "from:",ConnectionStart," to:",ConnectionEnd
end
```

Note

If you externally connect an *Interface* of a *Frame*, the *Interface* is passed to the parameter *predecessor/successor*.

The **standard connect control** as a *user-defined attribute* looks like this:



```

.M Models.MyPlantAnytown.MyStation1.OnConnect
1  param predecessor, successor, connector: object
2
3  -- ?: the object the control was called for
4
5  if connector /= void
6      -- Connection was created
7  else
8      -- Connection was deleted
9  end
10
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyStation.ConnectCtrl := &myConnectCtrl
```

See also

[Connect Control](#)

[Destructor Control](#)

ConstructorCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* when you duplicate, derive, or instantiate the object designated by <Path> (i.e., insert it into a *Frame* or a simulation model).

Type

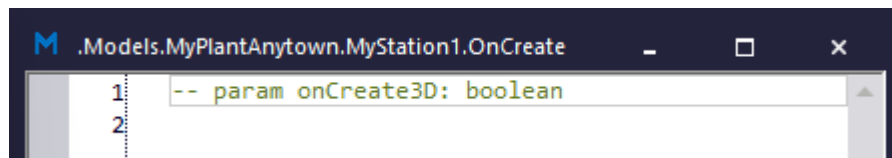
Attribute

Syntax

```
<Path>.ConstructorCtrl:method
```

Parameter

If you declare the **Constructor Control** with the optional parameter *onCreate3D* of data type *boolean*, the **Constructor Control** will not only be called for the events described above with *false*, but a second time when the 3D part of an object is created. In this case the parameter is *true*.



Note

The object already exists, when *Plant Simulation* calls the **Constructor Control**.

Assignment Value

You can assign a value of data type *method*.

Example

```
MyStation.ConstructorCtrl := &myConstructorMethod
```

See also

[Constructor Control](#)

DestructorCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* when you delete the object designated by <Path>. When a *Frame* contains *Sub-Frames* and objects, *Plant Simulation* will call **Destructor Controls** in these *Frames* as well.

When the *Plant Simulation* calls the *control* of the object, the object still exists.

Note

Plant Simulation calls a **Connect Control** before it calls a **Destructor Control**.

Type

Attribute

Syntax

```
<Path>.DestructorCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyStation.DestructorCtrl := &myDestructorMethod
```

See also

[Destructor Control](#)

[Connect Control](#)

DragDropCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* for the object designated by <Path>, which you drag onto another object and drop there. The dropped object can be an object or text.

Note

You cannot use the **Drag-and-drop Control** in the *Class Library* by dragging and dropping an object onto another object. The **Drag-and-drop Control** only works when you drag an object onto another object in a *Frame* and drop it onto another object there.

Type

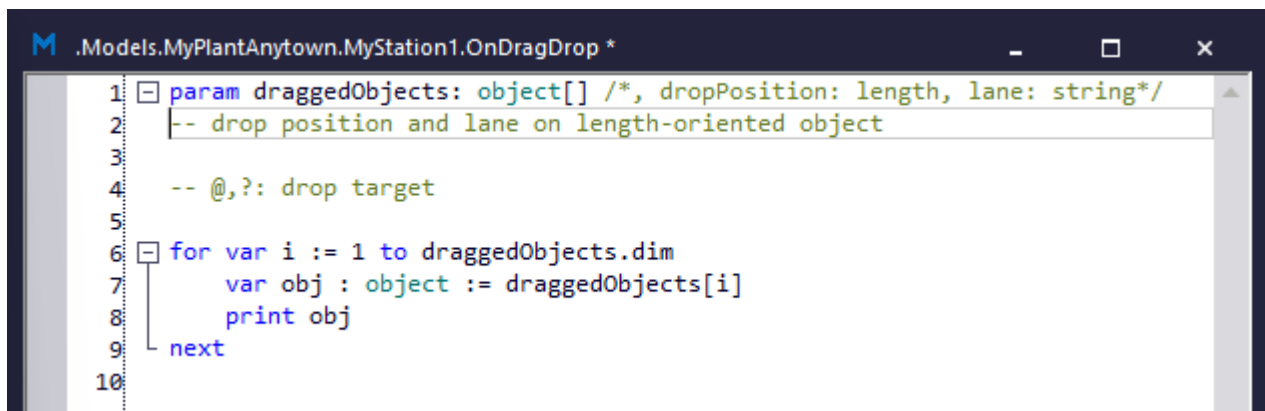
Attribute

Syntax

```
<Path>.DragDropCtrl:method
```

Parameter for point-oriented objects

The **standard drag-and-drop control** for point-oriented objects looks like this:



```
.Models.MyPlantAnytown.MyStation1.OnDragDrop *
1  param draggedObjects: object[] /*, dropPosition: length, lane: string*/
2    -- drop position and lane on length-oriented object
3
4    -- @,?: drop target
5
6    for var i := 1 to draggedObjects.dim
7      var obj : object := draggedObjects[i]
8      print obj
9    next
10
```

The parameter *draggedObjects* is an array of data type *object* that designates the path of the dragged and dropped objects.

Parameters for length-oriented objects

The **standard drag-and-drop control** for length-oriented objects looks like this:

```

1 param draggedObjects: object[] /*, dropPosition: length, lane: string*/
2   -- drop position and lane on length-oriented object
3
4   -- @,?: drop target
5
6 for var i := 1 to draggedObjects.dim
7   var obj : object := draggedObjects[i]
8   print obj
9 next
10

```

- The parameter *draggedObjects* is an array of data type *object* and designates the path of the dragged and dropped objects.
- *Plant Simulation* sets the data types of the optional parameters for length-oriented objects when the **Drag-and-drop Control** is called.
 - For a *length-oriented object* the optional parameter *dropPosition* of data type *length* designates the position starting at the beginning of the *length-oriented object* at which the object was dropped.
 - For the *TwoLaneTrack* the optional parameter *dropLane* of data type *string* designates the lane on the *TwoLaneTrack* on which the mouse button was released when dropping the object.

Drag-and-drop Control for Sensors

A **Drag-and-drop Control** for creating sensors on a *length-oriented object* and on a *TwoLaneTrack* might look like this.

```

param droppedObjects: object[], dropPosition: any := void, lane: any :=
void
// @,?: drop target

if lane = void
  if dropPosition /= void // length-oriented object
    ?.createSensor(dropPosition, "Length", void, true, false)
  end
else // TwoLaneTrack
  if lane = "A"
    ?.A.createSensor(dropPosition, "Length", void, true, false)
  else
    ?.B.createSensor(dropPosition, "Length", void, true, false)
  end
end
end

```

Assignment Value

You can assign a value of data type *method*.

Examples

```
MyStation.DragDropCtrl := &myDragAndDrop
```

```
param draggedObjects: object[]  
// one-dimensional array with n objects  
// the size of which can change  
var obj: object  
// @,?: drop target  
for var i := 1 to draggedObjects.dim  
    obj := draggedObjects[i]  
    // enter your source code here  
    print i, ": ", draggedObjects[i]  
    // prints the dragged objects to the Console  
next
```

See also

[Drag-and-Drop Control](#)

[Use a Drag-and-Drop Control for Several Objects](#)

FailCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* whenever the **Failed**-state of the object designated by <Path> changes. You can change the state in the dialog box of the object by selecting  **Failed** or by assigning the value `true` or `false` to the attribute *Failed*.

Within the *control* you can use the anonymous identifier question mark **?** to access the object which triggered the control.

Type

Attribute

Syntax

```
<Path>.FailCtrl:method
```

Parameters

Plant Simulation passes two optional parameters to the *Method* that is called:

- The optional parameter *FailureStartEnd* of data type *boolean* indicates if the failures starts (true) or ends (false).
- The optional parameter *FailureProfileName* of data type *string* designates the name of the *failure profile* that caused the failure.

Note

If the **Fail Control** declares both parameters, it will not only be called when the object changes to the **Failed**-state or when the **Failed**-state ends, but also each time when a *failure profile* is failed or when the failure is removed.

Let's say *failure profile 1* is already failed, and then you also fail *failure profile 2* in addition, then the **Fail Control** will still be called, although the object remains in the **Failed**-state.

Note

A **Fail Control**, which does not expect a parameter or only expects a single parameter, will only be called if the state of the object actually changes, meaning it will not be called when another *failure profile* is already failed. A **Fail Control**, which expects the name of the *failure profile* as the second parameter, will, by contrast, always be called when a *failure profile* is failed or when the failure is removed.

Assignment Value

You can assign a value of data type *method*.

Example

```
.MUs.Transporter.FailCtrl := &myFailCtrl
```

```
if ?.failed
    if ?.occupied and (NOT ?.cont.finished)
        ?.imp.releaseExporters
    end
else
    if ?.occupied and (NOT ?.cont.finished)
        ?.imp.import
    end
end
```

See also

[Fail Control \[material flow objects\]](#)

[Failed \[state - material flow objects\]](#)

[Failed \[check box\] - material flow objects](#)

[Failed \[check box\] - material flow objects](#)

InitCtrl [SimTalk] - general description

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* once at the beginning of the simulation run during the *init* phase. It is called before the object designated by <Path> is initialized and before the *init* controls are being executed.

Within the **Init Control** you can, for example, initialize the *Transporter* with suitable values and assign user-defined attributes or change the **Workers to Create table** of the *WorkerPool* for the next simulation run. This cannot be accomplished in an *init method* as this method would be called too late in time.

You can also initialize attributes which affect event generation, for example the **Availability**. Here the **Init Controls** are executed before events are computed. The normal *init methods* are executed after the initial events have been computed.

Type

Attribute

Syntax

```
<Path>.InitCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
.MUs.Transporter.InitCtrl := &myInitCtrl
```

```
M1.setupFor("C", false)
M2.setupFor("B", false)
M3.setupFor("A", false)
```

```
M1._3D.getObject("SetupType").setGraphicMaterial([1,
~,UserObjects.C._3D.MaterialDiffuseColor, 0, 0, 0, 0, 0)
M2._3D.getObject("SetupType").setGraphicMaterial([1,
~,UserObjects.B._3D.MaterialDiffuseColor, 0, 0, 0, 0, 0)
M3._3D.getObject("SetupType").setGraphicMaterial([1,
~,UserObjects.A._3D.MaterialDiffuseColor, 0, 0, 0, 0, 0)
```

SimTalk

[init \[SimTalk\] - predefined name](#)

See also

[Init Control \[general description\]](#)

[Init Control \[Transporter\]](#)

[Init Control \[EventController\]](#)

[Init Control \[WorkerPool\]](#)

[Workers to Create](#)

MoveInFrameCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* when the object designated by <Path> actually changes the position in the *Frame* into which you inserted it, meaning that it is moved within its *Frame*.

Within the *control* you can access the moved object with the anonymous identifier question mark ?.

Type

Attribute

Syntax

```
<Path>.MoveInFrameCtrl:method
```

Parameters

Plant Simulation passes two parameters to the *Method* that is called:

- The parameter *old* is an *array* of data type *length* with three parameters. They designate the x-coordinate, the y-coordinate, and the z-coordinate of the previous position of the object in the *Frame*.
- The parameter *new* is an *array* of data type *length* with three parameters. They designate the x-coordinate, the y-coordinate, and the z-coordinate of the new position of the object in the *Frame*.

Assignment Value

You can assign a value of data type *method*.

Examples

```
MyStation.MoveInFrameCtrl := &myMoveInFrameCtrl
```

```
param old, new: length[3]
print "xv:",previous[1]," yv:",previous[2]," zv:",previous[3],
      "xn:",new[1]," yn:",new[2]," zn:",new[3]
```

See also

Move in Frame Control

MoveToFolderCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* when you move the object designated by <Path> from a folder or a *Frame* to a different folder.

Within the *control* you can access the moved object with the anonymous identifier question mark ? and the new location with the anonymous identifier @.

Type

Attribute

Syntax

```
<Path>.MoveToFolderCtrl:method
```

Parameters

Plant Simulation passes three parameters to the *Method* that is called:

- The parameter *oldFolder* of data type *string* designates the previous location (*Frame* or folder) of the object.
- The parameter *newFolder* of data type *string* designates the new location of the object.
- The parameter *manually* of data type *boolean* shows the user if the object is manually moved to a folder (true) or with a *Method* (false).

Assignment Value

You can assign a value of data type *method*.

Examples

```
MySource.MoveToFolderCtrl := &MoveToFolderCtrl
```

```
param oldFolder, newFolder: string, manually: boolean
// @: new folder
// ?: moved object
// ?: the object for which the control was called
```

See also

[Move to Folder Control](#)

NotAvailableCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* whenever the state of the *Exporter* or the *Worker* designated by <Path> changes to **not available**.

Within the *control* you can use the anonymous identifier ? to access the object that triggered the *control*.

Type

Attribute

Syntax

```
<Path>.NotAvailableCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyExporter.NotAvailableCtrl := &myNotAvailableCtrl
```

See also

[Not Available Control \[Exporter\]](#)

OpenCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation does not open the dialog of the object though, but calls and runs the source code of the **Open Control**. Opening the dialog box is now completely controlled by the source code you specified.

Note

If *Plant Simulation* cannot open the dialog of the object any longer because of a faulty *Method*, create another *Method* and open the object from there with the method **openDialog**.

If you use a *user-defined attribute* of data type *method* as the **Open Control**, and if this *Method* is encrypted, you cannot open the respective object any longer. Only the **Open Control** itself, and any other *Method* that has been encrypted with the same password as the **Open Control**, can open the object.

If you specified an **Open Control** for a subtable, and if you then try to open this subtable, *Plant Simulation* runs the control without opening the subtable. This also applies to tables that are part of the object *Variable*. This also applies to an object of data type *table* which you inserted into a *Frame*.

To prevent the **Open Control** from being executed, hold down **Alt** while double-clicking the icon of the object. Instead, you can also select the command **Open Without Control** on the context menu of the object in the *Frame*.

When you click the button **Open Location**  or **Open Origin**  on the **Home Ribbon Tab**, you can hold down the **Alt** key while you click the button to prevent the **Open Control** of the class or of the origin to be executed respectively.

Note

To prevent the **Open Control** from being bypassed by holding down the **Alt** key, you can use an encrypted *user-defined attribute of data type method* as the **Open Control**.

If you want to remove the **Open Control**, you have to decrypt the *Method* beforehand. As you cannot open the object in the regular way, you can accomplish this with one of these procedures:

- Decrypt the model as a whole, this also decrypts all **Open Controls**.
- Click **Stop on Controls** on the **Debugger** ribbon tab and then double-click the object. This opens the *Debugger* for the **Open Control**. Select the menu command **Tools > Decrypt**.
- Call the method **decrypt**. You might, for example, specify `Frame5.OnOpen.decrypt("MyPassword123")`

Note

If you specify a *user-defined attribute* of data type *method* which you encrypted, you cannot delete or rename the attribute any longer. You have to decrypt the *Method* beforehand.

Note

If you open an object from the *Method Debugger* window, *Plant Simulation* will not execute the **Open Control**. This applies, for example, when you select **Open Object** on the context menu, or when you click **Open Location**  or **Open Origin** .

Type

Attribute

Syntax

```
<Path>.OpenCtrl:method
```

Parameters for all Objects Except for DataList and DataTable

You can declare these parameters:

- If you declare the **Open Control** with the parameter *from3D* of data type *boolean*, the value of the parameter will be set to true, if the object is opened from within a 3D window.
- If you declare the **Open Control** with the parameter *inLockedFolder* of data type *boolean*, the value of the parameter will be set to true, if the object is located within a locked library or a locked folder. If you do not declare the parameter, i.e., the **Open Control** expects none or only one parameter, and if the object to be opened is located within a locked library or a locked folder, the **Open Control** will not be called. Instead, *Plant Simulation* shows an error message letting you know that you are not allowed to modify the object.

Assignment Value

You can assign a value of data type *method*.

Examples

```
MyStation.OpenCtrl := VOID // delete entry
MyParallelStation.OpenCtrl := &myOpenControl
```

```
param from3D: boolean
var passwordHash:string := computeSHA1Hash(prompt("Password:"))
if passwordHash = "686483805ac47ca14e03514f7481a7973b401762"
```

```
// password "abc" specified?  
if from3D  
    ?._3D.openWindow  
else  
    ?.openDialog  
end
```

```
param from3D, inLockedFolder: boolean  
if inLockedFolder  
    if messageBox("The object is inside a locked folder."+strChr(10)+"Do you  
want to open the object anyway?", 48, 2) = 32 // "No"?  
        return  
    end  
end  
?.openDialog
```

See also

[Open Control](#)

PauseCtrl [SimTalk]

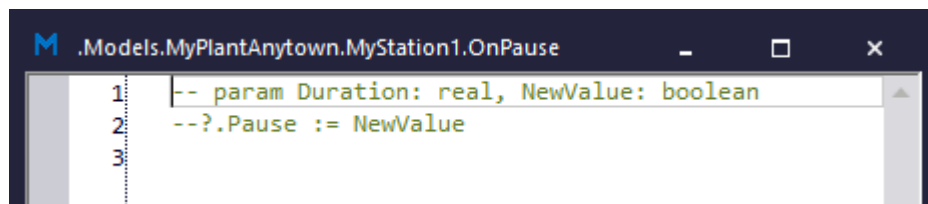
Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* whenever you change the **paused state** of the object designated by <Path>, when you either select an entry in the drop-down list **Paused/Planned/Unplanned**  or you assign another value to the attribute *Pause*.

When you get the **paused state** within the *Method*, it shows the state after the change.

The **standard pause control** as a *user-defined attribute* looks like this:



Type

Attribute

Syntax

```
<Path>.PauseCtrl:method
```

Parameters

You can specify the following parameters:

- The parameter *Duration* of data type *real* sets the length of the **pause**. If the length of the **pause** is unknown, *Plant Simulation* passes -1. The same happens when the control is called at the end of the **pause**.
- The parameter *NewValue* of data type *boolean* sets that the control will only be called if the *ShiftCalendar* changes the state of the object.

In addition, the object will not be put in the desired state. The **Pause Control** has to do this. The desired value is passed in the parameter *NewValue*.

The part still can be processed in its entirety before the object changes to the state **paused**.

Within the control you can access the paused object with the anonymous identifier **?**.

Note

If the **Pause Control** has the parameters listed above, then the *ShiftCalendar* will not automatically change the state **paused**. This is useful to complete processing of a part and then pause the shift manually by setting the attribute *Pause* of the object to **true**.

Compare this example:

```
param Duration: real, NewValue: boolean
if ?.Cont /= void
    wait ?.Cont.RemainingProcTime
end
?.Pause := NewValue
```

Note

If the **Pause Control** does not have any parameters, *Plant Simulation* always calls the **Pause Control** when the state **paused** changes.

If the **Pause Control** has the parameters listed above, *Plant Simulation* only calls the **Pause Control** when the state **paused** is changed by the *ShiftCalendar*. This prevents that the **Pause Control** calls itself when the **paused state** is set manually.

Assignment Value

You can assign a value of data type *method*.

Examples

```
Transporter.PauseCtrl := &myPauseCtrl
```

```
print "Current pause ", current.pause
MyStation.pause := current.pause
var shift := root.ShiftCalendar.GetCurrShift
print "Current shift: ", shift

if not current.unplanned

if current.pause
    current.currIcon := "pause"
else
    current.currIcon := "working"
end
```

See also

[Paused \[state, material flow objects\]](#)

[Pause \[SimTalk\] - material flow objects](#)

[Pause Control \[material flow objects\]](#)

[Pause Control \[Transporter\]](#)

PermitDeleteCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Note

The **Permit Delete Control** only applies to the *Frame* and to the *Folder*.

Plant Simulation calls *Method* for the *Frame* or the *Folder* designated by <Path>, when you attempt to delete the object for which you programmed the *control*.

- If the *method* returns false, *Plant Simulation* does not delete the object.
- If the *method* returns true, *Plant Simulation* deletes the object and calls a **Destructor Control** method, provided you entered one.

Type

Attribute

Syntax

```
<Path>.PermitDeleteCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
Frame.PermiDeleteCtrl := &myPermitDeleteCtrl
```

See also

[Permit Delete Control](#)

[Destructor Control](#)

PlausibilityCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* when you click **OK** or **Apply** in the dialog of the object designated by <Path> or in the windows of list objects to apply the changes you made there.

At this point in time the *control* checks if the entries are plausible, or it interprets them.

Type

Attribute

Syntax

```
<Path>.PlausibilityCtrl:method
```

Parameters for Lists and Tables

You can declare these parameters:

- If the **Plausibility Control** has no parameters, *Plant Simulation* only calls it, when you close the *list* or *table* after making a **change**. *Plant Simulation* also considers loading a file into a *table* as a change. This way *Plant Simulation* can check the entry and display an error message if the entry was wrong.
- If the **Plausibility Control** has two optional parameters of data type *integer*, *Plant Simulation* also calls the control after you edited a cell.

The parameter *ColumnNumber* of data type *integer* designates the column number of the edited cell.

The parameter *RowNumber* of data type *integer* designates the row number of the edited cell.

When you close the list or table, which you changed, or when you make any changes in any cell, *Plant Simulation* calls the **Plausibility Control** with the values -1 for the column and -1 for the row.

Assignment Value

You can assign a value of data type *method*.

Example

```
MyStation.PlausibilityCtrl := &myPlausibilityCtrl
```

```
param column,row: integer
switch column
```

```
case 1
// column-specific action
end
```

See also

[Plausibility Control](#)

RelabelCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* whenever you change the **Label** Label: of the object designated by <Path>.

Plant Simulation first calls the *control* for the relabeled object and then for the inheriting objects.

Type

Attribute

Syntax

```
<Path>.RelabelCtrl:method
```

Parameters

Plant Simulation passes three parameters to the *Method* that is called:

- The parameter *CurrentLabel* of data type *string* designates the current label of the object.
- The parameter *NewLabel* of data type *string* designates the new label.
- The parameter *ManualRename* of data type *boolean* indicates if the object is relabeled by the user (true) or by assigning a value to the attribute *Label* (false).

Assignment Value

You can assign a value of data type *method*.

Example

```
MyStation.RelabelCtrl := void
```

```
param CurrentLabel, NewLabel: string, ManualRename: boolean
print currentLabel
print newLabel
print manualRename
```

See also

[Relabel Control](#)

[Label \[general description\]](#)

[Label \[SimTalk\]](#)

RenameCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* when you change the **Name** Name:  of the object designated by <Path>.

Plant Simulation calls the *control* first for the renamed object and then for the inheriting objects.

Type

Attribute

Syntax

```
<Path>.RenameCtrl:method
```

Parameters

Plant Simulation passes three parameters to the *Method* that is called:

- The parameter *CurrentName* of data type *string* designates the current name of the object.
- The parameter *NewName* of data type *string* designates the new name of the object.
- The parameter *ManualRename* of data type *boolean* indicates if the object is renamed by the user (true) or by assigning a value to the attribute *Name* (false).

Note

Plant Simulation does not call the control when you insert the object into a *Frame* and then assign it a different name.

Assignment Value

You can assign a value of data type *method*.

Example

```
MyStation.RenameCtrl := &myRenameCtrl
```

```
param CurrentName, NewName: string, ManualRename: boolean
print CurrentName
```

```
print NewName  
print ManualRename
```

See also

[Rename Control](#)

[Name \[general description\]](#)

[Name \[SimTalk\] - general description](#)

SelectCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* when you click the object designated by <Path> with the left mouse button. *Plant Simulation* then does not select the object.

Instead, the *control* you entered has to take the appropriate actions, for example by opening the object, or by confirming the selection with the attribute *Selected*, etc.

3D models also support the **Select Control**. *Plant Simulation* calls the *Method* when the user clicks the mouse button. You can prevent calling the method by holding down the **Ctrl** key when clicking the mouse button.

Type

Attribute

Syntax

```
<Path>.SelectCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyStation.SelectCtrl := .Tools.&mySelectCtrl
```

```
print @.Selected  
@.Selected := true
```

See also

[Select Control](#)

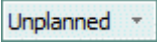
[_3D.Selected \[SimTalk\]](#)

UnplannedCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* whenever the **Unplanned state** of the object designated by <Path> changes, i.e., whether the object is not scheduled to work during the time, which you typed into the **ShiftCalendar** or whether it is scheduled to work.

You can change the state in the dialog of the object by selecting an entry in the drop-down list **Paused/Planned/Unplanned**  or by assigning another value to the attribute *Unplanned*.

Note

The **ShiftCalendar** uses the attribute *Unplanned* to designate the time period during which the object is not planned to work during a shift.

Type

Attribute

Syntax

```
<Path>.UnplannedCtrl:method
```

Parameters

You can specify the following parameters:

- The parameter *Duration* of data type *real* sets the length of the **shift pause**. If the length of the shift pause is unknown, *Plant Simulation* passes -1, as well as when the control is called at the end of the **shift pause**.
- The parameter *NewValue* of data type *boolean* sets that controls will only be called if the *ShiftCalendar* changes the state of the object.

In addition the object will not be put in the desired state. The **Unplanned Control** has to do this. The desired value is passed in the parameter *NewValue*.

The part still can be processed in its entirety before the object changes to the state **unplanned**.

Within the control you can access the object with the anonymous identifier ?.

Note

If the **unplanned control** has the parameters listed above, then the *ShiftCalendar* will not automatically change the state **unplanned**. This is useful to complete processing of a part and then end the shift manually by setting the attribute *Unplanned* of the object to `true`.

Compare this example:

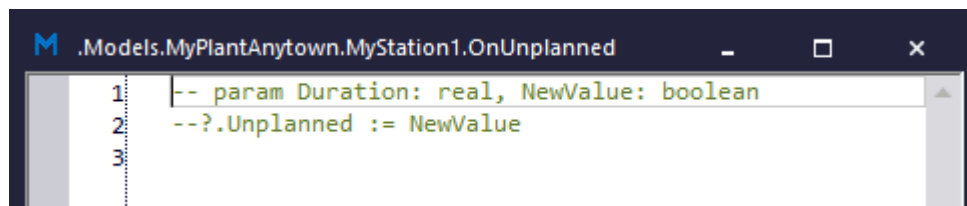
```
param Duration: real, NewValue: boolean
if ?.Cont /= void
    wait ?.Cont.RemainingProcTime
end
?.Unplanned := NewValue
```

Note

If the **Unplanned Control** does not have any parameters, *Plant Simulation* always calls the **Unplanned Control** when the state **unplanned** changes.

If the **Unplanned Control** has the parameters listed above, *Plant Simulation* only calls the **Unplanned Control** when the state **unplanned** is changed by the *ShiftCalendar*. This prevents that the **Unplanned Control** calls itself when the state **unplanned** is set manually.

The **standard unplanned control** as a *user-defined attribute* looks like this:

**Assignment Value**

You can assign a value of data type *method*.

Examples

```
MyStation.SelectCtrl := &myUnplannedCtrl
```

```
print "Frame unplanned: ", current.unplanned
MyStation.unplanned := current.unplanned
if current.unplanned
    current.currIcon := "unplanned"
else
    if current.pause
```

```

    current.currIcon := "pause"
else
    current.currIcon := "working"
end
end

```

See also

[Unplanned Control](#)

[Unplanned \[state, material flow objects\]](#)

[Unplanned \[SimTalk\] - material flow objects](#)

Attributes of User-defined Attributes

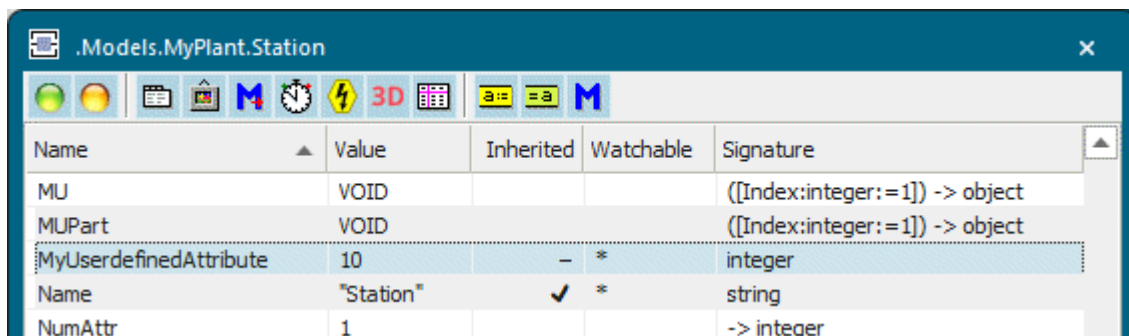
_Attributes of User-defined Attributes

The objects provide the *attributes* listed in the table of contents to the left for working with their *user-defined attributes*, which you define on the **tab User-defined**.

Plant Simulation addresses the attributes of *user-defined attributes* as sub-attributes of the respective *user-defined attribute*.

```
MyStation.myAttribute.Alignment := "Left"
```

The window **Show Attributes and Methods** shows the *user-defined attribute* itself and its value.



Name	Value	Inherited	Watchable	Signature
MU	VOID			([Index:integer:=1]) -> object
MUPart	VOID			([Index:integer:=1]) -> object
MyUserdefinedAttribute	10	-	*	integer
Name	"Station"	✓	*	string
NumAttr	1			-> integer

See also

[User-defined Attributes \[general description\]](#)

[Create a User-defined Attribute Manually](#)

[Create a User-defined Attribute During the Simulation](#)

_Methods of User-defined Attributes

Alignment [SimTalk] - user-defined attribute

Sets the **Alignment** of the specified *user-defined attribute* of the object designated by <Path> with its insertion point.

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.Alignment:string
```

Assignment Value

You can assign a value of data type *string*.

You can specify "Left", "Name", "Value", or "Right".

Example

```
MyStation.myAttribute.Alignment := "Left"
```

See also

[Alignment \[user-defined attribute\]](#)

asString [SimTalk] - user-defined attribute

Returns the value of the *user-defined attribute* as a string.

Remarks

Having the value as a string is especially useful for *user-defined attributes* of data type *object*. If the *user-defined attribute* contains a relative path, *asString* returns the relative path.

Type

Read-only attribute

Syntax

```
<Path>.<UserDefinedAttribute>.asString → string
```

Return Value

The return value has the data type *string*.

Example

```
Station5.ObjAttribute := "~.ProdMgr.prodplan"  
var a1 := Station5.ObjAttribute           //  
assigns .Models.Model.ProdMgr.prodplan  
var a2 := Station5.&ObjAttribute.asString // assigns "~.ProdMgr.prodplan"
```

See also

[User-defined Attributes \[general description\]](#)

BackgroundColor [SimTalk] - user-defined attribute

Sets the background color of the *user-defined attribute* of the object designated by <Path> in 3D.

Remarks

Set the RGB values of the color with the method *makeRGBValue*.

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.BackgroundColor:integer
```

Assignment Value

You can assign a value of data type *integer*.

Example

```
MyStation.myAttribute.BackgroundColor := makeRGBValue(100,100,100)  
MyStation.myAttribute.BackgroundColor := 6579300 // is the same as the  
color above
```

SimTalk

[makeRGBValue \[SimTalk\]](#)

[Color \[SimTalk\] - user-defined attribute](#)

Color [SimTalk] - user-defined attribute

Sets the color for displaying the values which the *user-defined attribute* of the object designated by <Path> visualizes in 3D.

Remarks

Set the RGB values of the color with the method *makeRGBValue*.

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.Color:integer
```

Assignment Value

You can assign a value of data type *integer*.

Example

```
MyStation.myAttribute.Color := makeRGBValue(0,255,0)
```

SimTalk

[makeRGBValue \[SimTalk\]](#)

See also

[Font Color \[user-defined attribute\]](#)

DataType [SimTalk] - user-defined attribute

Sets the data type of a non-inherited *user-defined attribute* of the object designated by <Path>.

Type

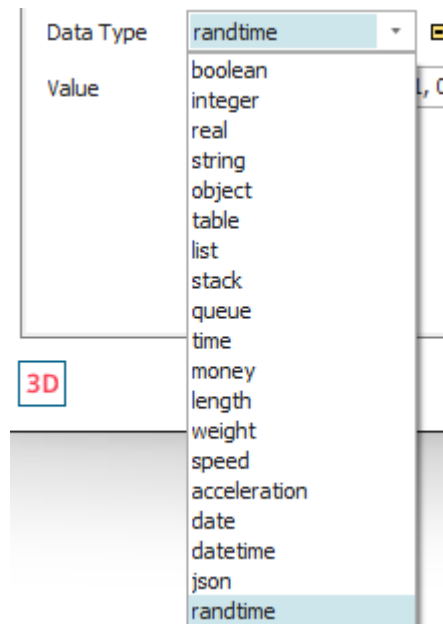
Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.DataType:string
```

Assignment Value

You can assign a value of data type *string*.



Example

```
MyStation.myAttribute.DataType := "String"
```

SimTalk

[getAttrType \[SimTalk\]](#)

[setAttrType \[SimTalk\]](#)

See also

[User-defined Attributes \[general description\]](#)

[New \[user-defined attribute\]](#)

DecimalPlaces [SimTalk] - user-defined attribute

Sets the number of decimal places which the specified *user-defined attribute* of the object designated by <Path> shows in 3D.

Remarks

The *user-defined attribute* can show up to 15 decimal places. The default setting **-1** shows all existing places.

Specify a positive value to show the number of decimal places, which you entered, in the *Frame*.

Note

This setting only applies to the **Data Types > real, length, money, weight, time, speed, and acceleration**.

The number of decimal places for the data type *time* affects values differently than for the data type *real*. The value **-1** for the data type *time* means that the default time display format is displayed with four decimal places. A positive value means that *Plant Simulation* shows the number of decimal places that you entered at most.

Type

Attribute

Syntax

```
Path.<UserDefinedAttribute>.DecimalPlaces:integer
```

Assignment Value

You can assign a value of data type *integer*.

Example

```
MyStation.myAttribute.DecimalPlaces := 8
```

SimTalk

& [SimTalk] - Variable

See also

[Data Type \[user-defined attribute\]](#)

[Decimal Places \[text box\] - Variable](#)

Font [SimTalk] - user-defined attribute

Sets the font size of the text of the *user-defined attribute* of the object designated by <Path> in 3D.

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.Font:integer
```

Assignment Value

You can assign a value of data type *integer*.

You can specify 1 for **Small**, 2 for **Medium**, 3 for **Large**, or 4 for **Extra Large**.

Example

```
MyStation.myAttribute.Font := 2
```

See also

[Font Size \[user-defined attribute\]](#)

HasInitValue [SimTalk] - user-defined attribute

Sets if the *user-defined attribute* of the object designated by <Path> has an initial value (**true**) or not (**false**).

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.HasInitValue:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStation.myAttribute.DataType := "integer"  
MyStation.myAttribute.HasInitValue := true
```

SimTalk

[inheritAttribute \[SimTalk\] - user-defined attribute](#)

[getAttribute \[SimTalk\] - user-defined attribute](#)

[setAttribute \[SimTalk\] - user-defined attribute](#)

See also

[Initial Value \[user-defined attribute\]](#)

InitValue [SimTalk] - user-defined attribute

Resets a value, which the *user-defined attribute* of the object designated by <Path> recorded during a simulation run.

Remarks

Plant Simulation resets the value during the **reset** and the **init phase** of the next simulation run.

Note

For the data types *table*, *list*, *stack*, *queue*, *randTime*, and *method* you cannot specify an **Initial Value**.

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.InitValue:any
```

Assignment Value

You can assign a value of data type *any*.

Example

```
MyStation.myAttribute.DataType := "integer"  
MyStation.myAttribute.InitValue := 12
```

SimTalk

[HasInitValue \[SimTalk\] - user-defined attribute](#)

[inheritAttribute \[SimTalk\] - user-defined attribute](#)

[getAttribute \[SimTalk\] - user-defined attribute](#)

[setAttribute \[SimTalk\] - user-defined attribute](#)

See also

[Initial Value \[user-defined attribute\]](#)

IntegerPlaces [SimTalk] - user-defined attribute

Sets the number of integer places, which the *user-defined attribute* of the object designated by <Path> shows in 3D.

Remarks

The *user-defined attribute* can show up to 15 integer places.

The default setting **-1** means that *Plant Simulation* applies no minimum on the number of integer places.

Specify a positive value to fill up the space before the decimal point with blank spaces until the specified number of places has been reached if the *user-defined attribute* is shown in 3D.

Note

This setting only applies to the **Data Types > integer, real, length, money, weight, time, speed, acceleration, and randTime**.

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.IntegerPlaces:integer
```

Assignment Value

You can assign a value of data type *integer*.

Example

```
MyStation.myAttribute.IntegerPlaces := 6
-- The value "123" will show as " 123"
-- The value "123456" will show as "123456"
-- The value "1234567" will show as "1234567"
```

See also

[Decimal Places \[text box\] - Display](#)

[Data Type \[user-defined attribute\]](#)

Name [SimTalk] - user-defined attribute

Sets the name of a not-inherited *user-defined attribute* of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.Name
```

Assignment Value

You can assign a value of data type *string*.

Example

```
MyStation.myAttribute.Name := "MyNewName"
```

See also

[Tab User-defined](#)

[New \[user-defined attribute\]](#)

[Name \[user-defined attribute\]](#)

Position [SimTalk] - user-defined attribute

Sets the position of the *user-defined attribute* of the object designated by <Path>.

Remarks

The position does not apply to *MU instances* and to *Connector instances*.

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.Position:length[3]
```

Assignment Value

You can assign an *array* with three values of data type *length*.

The values specify the position on the **X**-axis, the **Y**-axis, and the **Z**-axis.

Example

```
MyStation.myAttribute.Position := [5,3,0]
```

See also

[Settings for the Position \[user-defined attribute\]](#)

Rotation [SimTalk] - user-defined attribute

Sets the rotation of the *user-defined attribute* of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.Scale:real/real[4]
```

Assignment Value

You can specify:

- A number which defines the rotation around the negative z-axis at the position at which you inserted the object.
- An *array* with four items, which defines the rotation around the specified axis (values 2 to 4) at the position at which you inserted the object.

If you get the attribute, *Plant Simulation* returns an *array* with four items.

Examples

```
var a : any := MyStation.myAttribute.Rotation
// returns an array of 4 values
```

```
MyStation.myAttribute.Rotation := 30
// rotates the user-defined attribute by 30 degrees
```

```
MyStation.myAttribute.Rotation := [45, 0, 1, 0]
// rotates the user-defined attribute by 45 degrees around the Y axis
```

See also

[Settings for the Rotation \[user-defined attribute\]](#)

Scale [SimTalk] - user-defined attribute

Sets the scaling of the *user-defined attribute* of the object designated by <Path>.

Remarks

Scaling does not apply to *Connectors*.

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.Scale:real/real[3]
```

Assignment Value

You can assign a value of data type *real* to achieve uniform scaling.

You can also assign an *array* with three values of data type *real* to use different scaling factors for the three axes.

Examples

```
MyStation.myAttribute.Scale := 1           // uniform scaling
```

```
MyStation.myAttribute.Scale := [1, 4, 9]   // different scaling factors for  
the x-axis, y-axis and z-axis
```

See also

[Settings for the Scale \[transformation\]](#)

ShowDataType [SimTalk] - user-defined attribute

Shows the data type of the *user-defined attribute* of the object designated by <Path> in the *3D Frame*, into which you inserted the object to which it belongs (true) or hides it (false).

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.ShowDataType:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStation.myAttribute.ShowDataType := false
```

See also

[Show Data Type \[user-defined attribute\]](#)

ShowExternally [SimTalk] - user-defined attribute

Shows the *user-defined attribute* of the object designated by <Path> in the outside representation of the object (`true`) or not (`false`).

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.ShowExternally:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Specify `false` to show the *user-defined attribute* in the inside representation of the object to which it belongs. This is the view that *Plant Simulation* shows when you open the object with the commands **Open in 3D** or **Open**.

Example

```
MyStation.myAttribute.ShowExternally := false
```

See also

[Show Externally](#)

[Open in 3D \[object class\]](#)

[Open \[simulation object\]](#)

ShowIn3D [SimTalk] - user-defined attribute

Shows the *user-defined attribute* of the object designated by <Path> in 3D (`true`) or hides it (`false`).

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.ShowIn3D: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStation.myAttribute.ShowIn3D := true
```

See also

[Show in 3D \[user-defined attribute\]](#)

ShowInTooltip [SimTalk]

Shows the **Name** and the **Value** of the *user-defined attribute* in the tooltip of the object, when you hover the mouse of the object.

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.ShowInTooltip:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStation.myAttribute.ShowInTooltip := true
```

See also

[Show in Tooltip](#)

[Name \[SimTalk\] - user-defined attribute](#)

[setAttrValue \[SimTalk\]](#)

ShowName [SimTalk] - user-defined attribute

Shows the name of the *user-defined attribute* of the object designated by <Path> (`true`) or hides it (`false`).

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.ShowName: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStation.myAttribute.ShowName := true
```

See also

[Show Name \[user-defined attribute\]](#)

ShowUnit [SimTalk] - user-defined attribute

Sets if the *user-defined attribute* of the object designated by <Path> shows the units of the values (`true`) or not (`false`) in the *Frame* into which you inserted it.

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.ShowUnit: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStation.myAttribute.ShowUnit := true
```

See also

[Show Units \[user-defined attribute\]](#)

StatisticsActive [SimTalk] - user-defined attribute

Sets if the *user-defined attribute* of the object designated by <Path> collects statistics values (`true`) or not (`false`).

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.StatisticsActive:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStation.myAttribute.StatisticsActive := true
```

See also

[User-defined Attributes \[general description\]](#)

[Active \[check box\] - user-defined attribute](#)

Transparent [SimTalk] - user-defined attribute

Makes the background of the *user-defined attribute* of the object designated by <Path> transparent in 3D (true) or not transparent (false).

Type

Attribute

Syntax

```
<Path>.<UserDefinedAttribute>.Transparent:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Specify true to show the space around the text in the color of the background of the *Frame*.

Specify false to show the space around the text in white.

Example

```
MyStation.myAttribute.Transparent := true
```

See also

[Transparent \[user-defined attribute\]](#)

Methods of the Material Flow Objects

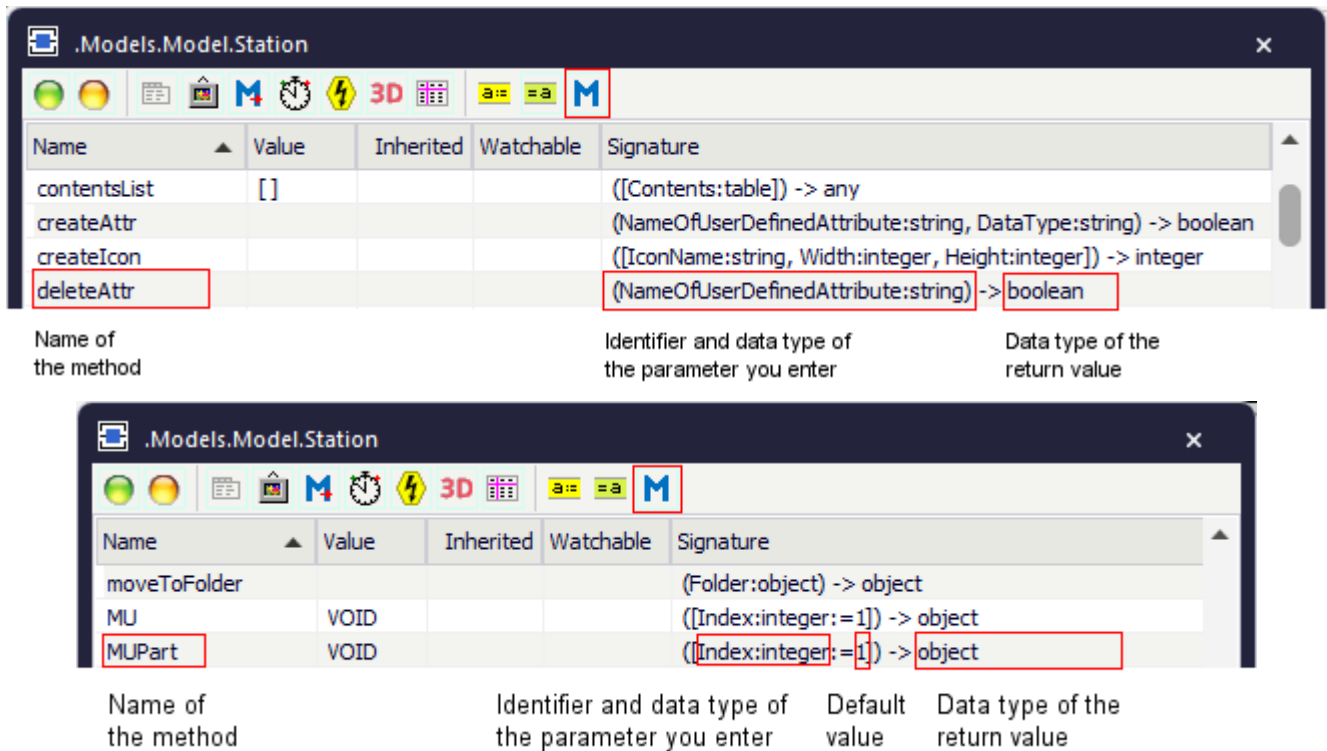
Methods of the Material Flow Objects

All material flow objects share the *methods* described below. The sub-chapters about the objects list additional methods for these objects.

A method

- gets information from an object and returns a value.
- computes a value.
- starts one or several actions that control the behavior of the object.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class** [general description].
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance** [general description].

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

Note

Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.